실전반 1회차 — 학습노트 실습 대본

[운영 · 비낭독] 강사가 혼자 자료를 읽고 조작·판단·요청·검토를 시범 보이는 발화다. 학생 응답이나 조작을 기다리지 않는다. 이번 문서 제작에서는 실습·화면 조작·캡처를 하지 않는다. 화면은 가이드의 동일 ID 자리에 나중에 넣는다. 아래 조작 안내와 미래형 발화는 수행 완료 기록이 아니다. 운영 시간은 미확정이다. 이전 고정 견적은 철회했다. 실제 화면에 맞는 조작을 확인한 뒤 해당 발화를 사용한다. 접이식 요청 예시는 실제 AI 대화가 아니며, 실제 응답은 관측한 내용에 따라 검토한다. 설치·가입·권한 복구는 별도 준비 시간이다.

[진행 원칙 · 비낭독] 조작하면서 해당 판단을 설명한다. 힌트와 AI 요청 예시는 필요한 경우에만 사용하며 모두 순서대로 소진하지 않는다. 이미 확인한 결론을 모르는 척하거나 질문 횟수를 채우지 않는다. 오류 분기는 실제 오류가 있을 때만 사용한다.

시작. 자료를 받아 작업 폴더 열기

[화면 01-START] 실습 자료 다운로드 → 압축 해제 → VS Code 폴더 열기 → 탐색기. 화면은 후속 삽입 자리다. 다운로드 링크는 배포한 01일차_실습1_학습노트.zip을 연결한다. 실제 실습·캡처는 이번 문서 제작에서 수행하지 않는다.
[조작] 이전 실습 파일을 닫는다. 수업 자료의 해당 ZIP을 내려받고 파일 탐색기에서 압축을 푼다. VS Code를 열어 파일 → 폴더 열기로 01일차_실습1_학습노트를 선택한다. 출처를 확인한 자료만 작업 공간 신뢰 여부를 판단한다. 파일 하나만 열거나 ZIP 내부에서 편집하지 않는다.

이번 실습 자료부터 받겠습니다. 같은 회차에도 실습이 여러 개 있으니까 파일 이름을 먼저 확인하겠습니다. 다운로드한 파일을 열어 보면 압축된 묶음입니다. 이 안에서 바로 수정하지 않고, 작업할 위치에 압축을 풀겠습니다.

이제 VS Code를 열겠습니다. 파일에서 폴더 열기를 선택하고, 방금 압축을 푼 폴더로 이동합니다. 이번 실습 폴더를 선택하겠습니다. 이전 실습 폴더와 섞이지 않도록 이름을 확인합니다.

왼쪽 탐색기에 파일 목록이 보입니다. 이 폴더가 지금 작업할 공간입니다. 파일 하나만 열었을 때와 달리, 자료와 결과를 같은 위치에서 관리할 수 있습니다. 학습노트_입력틀.md부터 열겠습니다. 이 양식을 따라 오늘 남길 결과와 확인할 항목을 먼저 보겠습니다.

[조작] 읽기 전용 입력과 수정할 파일, 결과가 저장될 위치를 차례로 짚는다. 전체 경로에 개인 사용자 이름이 보이면 화면에서 제외한다.

이 양식에는 설명과 출처, 검토 내용을 넣을 자리가 마련되어 있습니다. 우선 항목을 읽고 왜 필요한지 생각해 보겠습니다. 설명만 남겨도 될 것 같은데 출처와 수정 이유까지 필요한 이유는 무엇일까요? 다시 찾아보고 고치려면 처음 생각과 확인한 근거가 남아 있어야겠네요. 이 틀 안에 어떤 내용을 넣을지는 제가 판단하겠습니다.

1. 결과와 준비 확인

[화면 01-01] 수업용 작업 공간과 과정 홈의 세 영역. 개인 페이지·계정 정보는 제외한다.
[조작] 편집할 수 있는 작업 공간을 준비하고 과정 홈에서 개념·참고 자료·실습 결과로 이동할 구성을 먼저 적는다. 설치·가입·권한 복구가 필요하면 별도 준비 시간으로 기록한다.

[조작 · 비낭독] Notion을 열어 수업용 작업 공간을 선택한다. 새 빈 페이지를 만들고 제목을 입력한다. 기성 완성 노트를 복제하지 않는다. VS Code의 입력틀은 옆에 두고 필요한 항목만 차례로 옮긴다.

이제 Notion을 열겠습니다. 수업 기록을 둘 작업 공간에서 빈 페이지를 만들고, 과정 이름을 제목으로 적겠습니다. 아직 내용이 없는 상태입니다. 왼쪽에 둔 입력틀을 통째로 복사하기보다, 기록을 어디에서 찾고 어떻게 확인할지 생각하면서 필요한 부분부터 만들겠습니다.

이번에는 방금 배운 내용을 다시 찾고 고칠 수 있는 노트를 만들겠습니다. 첫 주제는 메모와 지식 자산의 차이입니다. 제가 이해한 것을 제 말로 적고, 그 설명을 확인할 자료를 연결하는 것이 목표입니다.

기록 제목을 정하면서 생각해 보겠습니다. 다음 주에 이 내용을 잊었다면 어떤 말로 검색할까요? 저는 개념 이름부터 찾을 것 같습니다. 수업 이름만 적으면 그 안의 어떤 내용을 찾는지 알기 어렵습니다. 과정 홈은 전체 기록으로 들어가는 입구로 두고, 각 기록에는 찾으려는 개념의 이름을 붙이겠습니다.

기록을 다시 쓸 때 하는 일을 떠올려 보겠습니다. 개념 설명을 읽고, 근거 자료를 열고, 실제로 적용한 결과를 찾겠죠. 그래서 개념, 참고 자료, 실습 결과의 세 영역을 두겠습니다. 틀에 적혀 있어서 옮기는 것이 아니라 이 세 가지 일을 연결하기 위한 구성입니다. 제출할 때 누가 읽을 수 있어야 하는지도 수업에서 정한 범위로 확인하겠습니다.

2. 과정 홈과 기록 틀 만들기

[화면 01-02] 과정 홈, 학습노트 속성, 같은 기록의 표·이해도별 보기.
[조작] 홈과 하위 영역을 만들고 제목·본문 블록·목차·링크로 연결한다. 학습노트 데이터베이스에 제목·분야·이해도·작성일을 둔다. 같은 기록을 표와 이해도별 보기에서 확인한다.

과정 홈 아래에 학습노트 목록을 만들겠습니다. 기록마다 제목, 분야, 이해도, 작성일을 붙입니다. 이해도는 잘 보이게 꾸미는 항목이 아니라, 무엇을 다시 공부해야 하는지 찾는 기준입니다. 아직 설명하기 어렵다면 그 상태를 표시해 두겠습니다.

목차와 링크도 넣겠습니다. 목차는 이 페이지 안의 대목을 찾는 데, 링크는 연결한 다른 기록이나 자료로 가는 데 쓰겠습니다. 개념을 설명하다가 참고 자료로 이동하고, 실습 결과에서 그때 사용한 개념을 다시 찾아올 수 있게 연결하겠습니다.

같은 기록을 표로도 보고 이해도에 따라 모아서도 볼 겁니다. 보기마다 내용을 복사하면 수정한 내용이 서로 달라질 수 있겠죠. 같은 기록을 다른 기준으로 보여 주도록 구성하겠습니다. 한 보기에서 기록을 열고 다른 보기에서도 같은 기록인지 확인하겠습니다.

3. 자기 말로 첫 기록 쓰기

[화면 01-03] 첫 설명, 교재 출처, 수정했다면 전후 문장과 이유.
[조작] 교재를 잠깐 덮고 첫 설명을 쓴 뒤 교재 4~9장과 대조한다. 강사 설명→Copilot 질문→강사 자기 답변→응답 검토→필요한 문장 수정 순으로 진행한다. 실제 응답이 없는 상태에서 답변을 만들어 읽지 않는다.

첫 제목은 메모와 지식 자산의 차이로 하겠습니다. 자료를 잠깐 덮고 두세 문장으로 적겠습니다. 문장을 길게 쓰는 것보다 내가 다시 사용하려면 무엇이 필요하다고 이해했는지가 중요합니다. 아직 배우지 않은 기술을 미리 설명할 필요는 없습니다.

제가 처음 적을 설명은 이렇습니다. 메모는 그때 본 내용을 남기는 기록이고, 지식 자산은 나중에 찾아 다시 쓰고 고칠 수 있도록 설명과 근거를 연결한 기록이라고 정리하겠습니다. 이 설명이 교재의 뜻과 맞는지 검토하겠습니다.

이제 교재를 열어 비교하겠습니다. 제가 단정한 표현이 원문에도 있는지, 중요한 조건을 빠뜨리지 않았는지 확인하겠습니다. 문장이 술술 읽혀도 근거와 다를 수 있습니다. 고칠 부분이 있다면 첫 설명을 남겨 두고, 무엇을 왜 바꿨는지 기록하겠습니다.

설명은 이 Notion 기록에 남기겠습니다. 제가 쓴 문장과 교재의 근거 대목만 Copilot 채팅에 붙여, 다시 사용할 때 빠진 조건이 없는지 검토하겠습니다. 입력틀은 항목을 확인하는 참고로 두고 같은 기록을

두 군데 관리하지 않겠습니다.

[조작 · 비낭독] Notion의 첫 설명과 근거 위치를 필요한 범위만 채팅에 붙인다. 교재 발췌는 실제 원문을 사용한다. 입력 틀에 같은 내용을 다시 작성하지 않는다.
[입력 · 비낭독] 제가 적은 설명과 교재 대목을 비교하려 합니다. 먼저 제가 어떤 상황에서 이 기록을 다시 쓰려는지 질문 하나를 해 주세요. 제 답을 들은 뒤 원문과 다른 주장이나 빠진 조건이 있다면 그 문장만 검토해 주세요. 전체 노트는 만들지 마세요.
[응답 분기 · 비낭독] 실제 질문을 읽고 강사가 답한다. 재사용 상황을 묻는다면 아래 자기 답변을 사용한다. 질문의 내용이 다르면 그 내용에 맞춰 사실대로 답한다. 질문 없이 수정안을 내놓으면 적용 전에 자신의 사용 목적을 설명한다.

저는 다음 회차에서 앞 개념을 다시 확인할 때 이 기록을 쓰려 합니다. 제목으로 찾고, 제 설명을 읽은 뒤 근거 대목까지 돌아갈 수 있어야 합니다. 그래서 단순히 파일이 남아 있는 것보다 찾을 이름과 출처가 연결되어 있는지를 보겠습니다.

[입력 · 비낭독] 강사의 실제 답변을 채팅에 입력한다. 이어 받은 검토에서 원문에 없는 주장, 빠진 조건, 단순 표현 제안을 구분한다. 수정이 필요하면 해당 문장만 요청·검토하고 Notion의 해당 기록에 반영한다. 오류가 없으면 억지로 고치지 말고 대조한 근거를 남긴다.

받은 제안을 원문과 나란히 보겠습니다. 문장이 매끄러워졌더라도 교재의 뜻이 달라지면 채택하지 않겠습니다. 필요한 문장만 고치고 처음 설명과 수정 이유를 함께 남기겠습니다. 마지막 단계에서 이 기록을 다시 찾아 출처가 열리는지 확인하겠습니다.

4. 출처와 실습 기록 연결하기

[화면 01-04] 근거 위치가 적힌 참고 자료와 아직 수행하지 않은 실습의 결과 양식.
[조작] 교재 이름·장·판본과 확인한 대목을 기록한다. 실습 결과에 입력·기대·관측·문제·다음 확인 항목을 만든다. 수행하지 않은 항목은 미수행으로 둔다.

참고 자료를 연결할 때 링크만 남기면 긴 문서 안에서 근거를 다시 찾기 어려울 수 있습니다. 교재 이름과 판본, 장 번호, 어떤 대목을 확인했는지도 적겠습니다. 내가 쓴 설명에서 출처로 돌아갈 수 있어야 나중에 이해가 바뀌었을 때 고칠 수 있습니다.

실습 결과를 담을 자리도 마련하겠습니다. 무엇을 넣었는지, 무엇을 기대했는지, 실제로 무엇이 나왔는지 나누어 적는 겁니다. 기대와 관측이 다르면 문제와 다음 확인을 남깁니다. 아직 하지 않은 뒤 실습은 결과를 비워 두고 미수행이라고 표시하겠습니다.

실패를 겪지 않았다면 실패한 것처럼 채울 필요는 없습니다. 반대로 원인을 찾지 못했는데 해결했다고 적어서도 안 됩니다. 다음에 이어서 볼 사람이 어디까지 확인했는지 알 수 있게 남기겠습니다.

5. 찾기·보완·제출하기

[화면 01-05] 검색 결과, 낮은 이해도 보기, 출처 재열기와 제출 범위.
[조작] 과정 홈에서 제목으로 첫 기록을 찾고, 이해도가 낮은 기록을 골라 출처를 연 뒤 돌아온다. 수업에서 정한 열람 범위로 제출 링크를 확인한다. 확인하지 못한 항목은 결과를 채우지 않는다.

과정 홈으로 돌아가 방금 쓴 기록을 다시 찾아보겠습니다. 제목으로 찾고, 이해도가 낮은 항목만 모아 볼 수 있는지도 확인하겠습니다. 출처를 열었을 때 내 설명의 근거가 있는 대목으로 갈 수 있어야 합니다.

다른 페이지가 열리거나 권한이 없다면 기록을 쓴 것과 검토할 수 있는 것을 구분해야 합니다. 링크가 있다는 사실만으로 제출 준비가 끝난 것은 아니에요. 정해진 검토자가 읽을 수 있는지 확인하고, 확인할 수 없다면 열람 미확인으로 남기겠습니다.

제출할 것은 첫 설명, 확인한 출처, 고친 문장과 그 이유입니다. 고칠 문장이 없었다면 남은 질문이나 검토한 근거를 적겠습니다. 이어지는 실습 결과도 이 노트에 쌓겠습니다. 노트의 틀을 마련했으니, 다음 시간에는 설비 데이터가 판단으로 이어지는 시스템의 구조를 살펴보겠습니다.

[연결 · 비낭독] 이론 12장에서 재개한다. 화면과 관측 결과는 후속 촬영 때 같은 ID 자리에 삽입한다.

실전반 1회차 — 실행환경 실습 대본

[운영 · 비낭독] 강사가 혼자 자료를 읽고 조작·판단·요청·검토를 시범 보이는 발화다. 학생 응답이나 조작을 기다리지 않는다. 이번 문서 제작에서는 실습·화면 조작·캡처를 하지 않는다. 화면은 가이드의 동일 ID 자리에 나중에 넣는다. 아래 조작 안내는 수행 완료 기록이 아니다. 운영 시간은 미확정이다. 이전 고정 견적은 철회했다. 명령·접속 주소·실행 순서는 촬영에 쓰는 판본의 절차로 확인한다. 기존 사용자 환경을 초기화하지 않는다.

[진행 원칙 · 비낭독] 조작하면서 해당 판단을 설명한다. 힌트와 AI 요청 예시는 필요한 경우에만 사용하며 모두 순서대로 소진하지 않는다. 이미 확인한 결론을 모르는 척하거나 질문 횟수를 채우지 않는다. 오류 분기는 실제 오류가 있을 때만 사용한다.

시작. 자료를 받아 작업 폴더 열기

[화면 02-START] 실습 자료 다운로드 → 압축 해제 → VS Code 폴더 열기 → 탐색기. 화면은 후속 삽입 자리다. 다운로드 링크는 배포한 01일차_실습2_실행환경.zip을 연결한다. 실제 실습·캡처는 이번 문서 제작에서 수행하지 않는다.
[조작] 이전 실습 파일을 닫는다. 수업 자료의 해당 ZIP을 내려받고 파일 탐색기에서 압축을 푼다. VS Code를 열어 파일 → 폴더 열기로 01일차_실습2_실행환경을 선택한다. 출처를 확인한 자료만 작업 공간 신뢰 여부를 판단한다. 파일 하나만 열거나 ZIP 내부에서 편집하지 않는다.

이번 실습 ZIP을 내려받아 새 작업 폴더에 압축을 풀겠습니다. VS Code의 파일, 폴더 열기에서 그 폴더를 선택합니다. 탐색기에서 실습 이름을 확인하고 환경_확인표.md를 열겠습니다. 이번에 작성할 항목과 결과를 저장할 위치부터 보겠습니다.

[조작] 읽기 전용 입력과 수정할 파일, 결과가 저장될 위치를 차례로 짚는다. 전체 경로에 개인 사용자 이름이 보이면 화면에서 제외한다.

확인표에는 기대 상태, 관측 결과, 근거, 판정을 적을 자리가 있습니다. 아직 정상이라는 답은 적혀 있지 않습니다. 먼저 어떤 일을 하는 서비스인지 알아야 기대 상태를 정할 수 있겠네요. 제가 절차와 구성을 읽고 그 기준부터 채우겠습니다.

이 ZIP에는 실습 안내와 확인표가 있고, HYD 서비스 전체가 들어 있지는 않습니다. 환경 준비 안내를 열어 수업에 사용하는 시스템 저장소 위치를 확인하겠습니다. 자료 폴더와 실제 서비스를 실행할 폴더를 구분해 두겠습니다.

[조작 · 비낭독] 환경_준비_안내.md에서 별도 시스템 저장소 준비 조건을 확인한다. 새 VS Code 창의 폴더 열기로 수업용 HYD 저장소 루트를 연다. compose.yaml, ot, dmz, it, 판본별 실행 안내가 실제로 있는지 확인한다. 준비된 저장소가 없으면 제공되지 않은 경로를 임의로 말하거나 빈 실습 폴더에서 기동 명령을 실행하지 않는다. 수업용 저장소 확보를 사전 준비 항목으로 남긴다.

새 창에서 시스템 저장소의 맨 위 폴더를 열겠습니다. 여기서는 Compose 파일과 세 영역의 폴더를 확인합니다. 두 창을 열어 두고, 시스템 창에서는 구성을 읽고 확인한 결과를 실습 창의 표에 적겠습니다.

1. 판본과 시작 상태 기록

[화면 02-01] 판본, 폴더 구조, 기대 상태 표. 비밀 설정값은 제외한다.
[조작] 제공된 HYD 저장소의 판본·실행 구성을 기록하고 ot·dmz·it 폴더와 compose.yaml의 역할을 연결한다. 이미 켜진 환경이면 현재 상태부터 관찰한다.

이번에는 서비스를 실행하고 각 역할이 동작할 준비가 됐는지 확인하겠습니다. 같은 프로젝트라도 어떤 기능을 켜는지에 따라 필요한 서비스가 달라질 수 있습니다. 그래서 먼저 이번 환경의 판본과 실행 구성을 기록하겠습니다.

폴더에서 OT, DMZ, IT에 해당하는 부분을 찾겠습니다. 앞 이론에서 본 역할들이 어느 코드와 구성에 연결되는지 확인하는 겁니다. Compose 파일에는 함께 실행할 서비스의 구성이 있습니다. 그 밖에서 준비해야 하는 작업자나 연결도 있는지 제공 절차와 대조하겠습니다.

이름만 보고 모두 같은 상태여야 한다고 적어도 될까요? 우선 서비스 하나를 골라 시작 명령과 의존 대상을 읽겠습니다. 계속 요청을 받는 역할인지, 자료를 준비하고 끝나는 역할인지부터 확인하겠습니다. 그 뒤 기대 상태를 표에 적겠습니다. 포털이 열리는 것만으로 판단부터 조치까지 전체 과정이 준비됐다고 볼 수는 없습니다.

[선택 분기 · 비낭독] 구성과 절차를 읽어도 역할을 설명하기 어려운 항목이 있을 때만 아래 질문을 사용한다. 설명할 수 있으면 기대 상태를 적고 도구 확인으로 진행한다.
[입력 · 비낭독] 구조 탐색 요청: 첨부한 구성의 서비스 한 항목이 무슨 일을 하는지 파악하려 합니다. 아직 정상 여부를 판정하거나 설정을 수정하지 마세요. 명령·의존성·상태 검사 중 무엇부터 읽어야 할지 질문 하나로 안내해 주세요. 제가 해당 부분을 읽고 역할을 설명하겠습니다.
[응답 분기 · 비낭독] 강사가 선택한 항목의 실제 구성으로 답한다. 명령을 먼저 읽으라는 질문이면 구성에 적힌 명령과 역할을 확인하고, 다른 질문이면 그 위치를 찾는다. 자료가 없으면 서비스 역할을 추측하지 않는다. 이후 상시 서비스·일회성 준비·외부 의존성 중 해당 범위를 정리한다.

2. 도구와 엔진 구분

[화면 02-02] 도구 버전과 엔진 상태를 구분한 화면.
[조작] docker --version, docker compose version, python --version으로 각 도구의 응답을 확인한다. Docker 엔진 상태는 별도로 확인한다. Copilot 사용 여부는 필수 환경 판정과 구분한다.

VS Code는 파일을 읽고 터미널에서 명령을 실행할 작업 공간입니다. Docker Desktop은 컨테이너 실행 환경을 제공하고, Compose는 정의된 여러 서비스를 함께 다룹니다. Python은 다음 기록 추적 랩의 코드를 실행할 때 사용합니다.

버전 확인 명령부터 읽어 보겠습니다. 버전이 나오면 그 명령을 실행할 수 있다는 근거가 됩니다. 하지만 Docker 명령의 버전이 나왔다는 것만으로 엔진이 실행 중이거나 HYD 서비스가 준비됐다고 말할 수는 없습니다. 도구, 엔진, 서비스가 각각 어떤 상태인지 따로 확인하겠습니다.

AI의 도움을 사용하는 경우에도 관측을 먼저 남기겠습니다. 버전은 나오는데 다음 단계가 안 된다면, 무엇까지 확인했는지와 어디서부터 안 됐는지를 질문에 넣습니다. 제공된 절차와 코드를 따라가는 데 모델 키는 배포하지 않습니다. 선택적인 대화 사용 여부와 서비스의 정상 여부도 별개로 기록하겠습니다.

3. 실행 순서 읽고 적용 범위 정하기

[화면 02-03] 해당 판본의 실행 절차와 단계별 준비 확인. 토큰·키는 제외한다.
[조작] 문서/시스템/4_켜고_돌리는_법.md와 수업의 실행 구성을 대조한다. 업무 데이터베이스·서비스·외부 작업자의 선행 조건을 확인하고 해당 프로필의 기동 절차를 적용한다. 이미 실행 중이면 재기동 장면을 만들려고 내리지 않는다.

이제 제공 절차에서 이번 구성의 시작 순서를 찾겠습니다. 어떤 서비스가 다른 서비스의 준비를 기다리는지 살펴보겠습니다. 선행 조건이 빠지면 뒤의 서비스만 다시 켜도 문제가 풀리지 않을 수 있습니다.

환경 변수는 어떤 설정을 정하는지, 실행 프로필은 어떤 구성을 선택하는지 확인하겠습니다. 비밀값 자체를 기록하거나 보여 줄 필요는 없습니다. 사용한 설정의 이름과 구성, 문제가 생겼을 때 볼 로그 위치를 남기면 됩니다.

단계를 하나 진행할 때마다 무엇이 끝났는지 확인하겠습니다. 주소 목록이 출력된 것과 그 주소에서 필요한 응답을 받은 것은 다릅니다. 절차에 적힌 준비 조건을 확인한 뒤 다음 단계로 넘어가겠습니다. 이미지 내려받기나 설치, 권한 복구가 필요한 시간은 별도로 기록합니다.

4. 상태를 종류별로 판정

[화면 02-04] 장기 실행 서비스 상태, 일회성 작업의 종료 코드·로그, 기대와 관측의 비교.
[조작] 해당 구성에서 docker compose ps -a 등으로 상태를 확인한다. 로그는 서비스와 관측 시각이 맞는 필요한 구간을 사용한다. 실제 실패가 없다면 실패 사례를 연출하지 않는다.

상태 목록을 볼 때는 먼저 그 항목이 어떤 일을 하는지 확인하겠습니다. 계속 실행되는 서비스라면 실행 상태와 해당 서비스가 제공하는 상태 검사 결과를 봅니다. 자료를 준비하고 끝나는 일회성 작업이라면 종료 코드와 완료 기록을 봐야 합니다.

종료됐다는 표시만으로 실패라고 하면 정상적으로 일을 마친 준비 작업도 실패로 읽게 됩니다. 반대로 실행 중이라는 표시만으로 모든 기능이 정상이라고 하면 연결 문제를 놓칠 수 있겠죠. 기대 상태 옆에 실제 관측과 그 근거를 적어 보겠습니다.

확인표에서 기대와 관측이 다른 항목이 있는지 보겠습니다. 차이가 있다면 그 행과 관련 로그를 놓고 원인을 좁히겠습니다. 모두 기대와 맞으면 확인한 범위를 남기고 포털 접속으로 넘어가겠습니다. 분명한 결과를 다시 묻기 위해 대화를 만들 필요는 없습니다.

[선택 분기 · 비낭독] 아래 대화는 기대와 다른 관측이나 판단할 수 없는 항목이 있을 때만 진행한다. 정상 경로에서는 건너뛰고 §5로 이동한다.
[조작 · 비낭독] 환경_확인표.md에 실제 기대·관측·근거를 적고 채팅 컨텍스트로 선택한다. 관련 문서의 해당 판본 절차와 필요한 로그만 추가한다. 비밀 설정값은 첨부하지 않는다.
[입력 · 비낭독] 이 확인표에서 도구 설치, 엔진 실행, 서비스 준비를 구분하려 합니다. 제가 확인했다고 쓴 항목 하나의 근거를 먼저 질문해 주세요. 답한 뒤 아직 입증하지 못한 범위와 다음 확인 한 가지만 제안해 주세요. 관측하지 않은 장애를 추정하거나 전체 재설치·초기화를 제안하지 마세요.
[응답 분기 · 비낭독] 실제 질문을 읽는다. 강사는 해당 관측과 그 한계를 답한다. 버전 출력에 대한 질문이면 "버전 출력으로 확인한 것은 명령을 사용할 수 있다는 점이고, 엔진과 서비스는 별도 확인이 필요합니다."라고 답한다. 다른 질문이면 실제 확인표의 해당 근거로 답한다.

제 답변을 보낸 뒤 제안된 다음 확인이 원래 절차와 맞는지 보겠습니다. 질문에 대한 답이 그럴듯해도 이번 판본에 없는 명령이나 서비스를 가져오면 그대로 적용할 수 없습니다. 필요한 항목 하나만 골라 확인하겠습니다.

[조작·분기 · 비낭독] 실제 응답을 원문 절차와 대조한다. 기대·관측을 혼동한 표의 문장이 있다면 그 행만 수정한다. 시스템 변경은 해당 판본 절차에서 필요한 것으로 확인된 경우만 적용하며 초기화하지 않는다. 필요한 상태 조회나 접속을 한 가지 수행하고 관측을 갱신한다. 기대와 다르면 해당 출력으로 다음 확인을 좁히고, 같으면 그 범위만 확인으로 남긴다. 실제 장애가 없으면 정상 서비스와 일회성 작업의 판정 차이를 설명한다.

확인표를 고친 것과 환경 문제가 해결된 것은 다릅니다. 방금 확인한 결과를 근거로 판정을 남기고, 아직 확인하지 않은 기능까지 정상이라고 넓혀 적지는 않겠습니다.

원인 후보를 얻은 것과 해결한 것은 다릅니다. 고쳤다면 같은 기준으로 다시 확인한 결과가 있어야 합니다. 실제 실패 로그가 없다면 이번 실행에서 실패를 겪었다고 작성하지 않습니다.

5. 접속 결과와 미확인 구분

[화면 02-05] 포털 접속 결과와 환경 확인표. 미확인 항목도 남긴다.
[조작] 제공 주소의 포털과 주요 서비스를 각각 확인하고, 다음 전체 과정에 필요한 외부 작업자의 준비 상태를 구분한다. 접속하지 않은 것은 미수행, 시도했지만 필요한 증거를 얻지 못한 것은 미확인으로 적는다.

마지막으로 포털과 주요 서비스의 접속을 확인하겠습니다. 접속하지 못했다면 서버 장애라고 바로 결론 내리지는 않겠습니다. 주소가 맞는지, 권한이 있는지, 연결이 닿는지, 서비스가 어떤 응답을 줬는지를 나누어 보겠습니다.

확인표에는 기대 상태, 관측 상태, 근거 위치, 판정을 남깁니다. 목록에서 서비스가 몇 개 보였다는 것보다 이번 구성에 필요한 항목을 각각 확인했는지가 중요합니다. 외부 작업자를 아직 확인하지 못했다면 포털 접속 성공과 별도로 표시하겠습니다.

이 표를 바탕으로 다음 실습을 준비하겠습니다. 다음에는 저장된 사건의 처리 과정을 읽으면서 추천과 승인, 실행과 결과를 구별하고, 작업지시에서 판단의 근거를 거슬러 찾아보겠습니다.

[연결 · 비낭독] 전체 과정 관찰·기록 추적 실습으로 이어 간다. 실제 화면·관측 결과는 후속 삽입하며, 이번 문서 완성을 환경 실행 성공으로 보고하지 않는다.

실전반 1회차 — 전체과정 관찰·기록 추적 실습 대본

[운영 · 비낭독] 강사가 혼자 자료를 읽고 판단·요청·검토·수정·실행을 시범 보이는 발화다. 학생 응답이나 조작을 기다리지 않는다. 이번 문서 작성에서는 실행·화면 조작·캡처를 하지 않았다. 화면은 후속 삽입한다. 실제 Copilot 응답과 실행 출력은 아래 분기에 따라 읽으며 예시 답변을 실제 응답으로 대신하지 않는다. 운영 시간은 미확정이다. 이전 고정 견적은 철회했다. 기존 완성본의 검증 결과는 시작본의 구현 성공을 보증하지 않는다.

[진행 원칙 · 비낭독] 조작하면서 해당 판단을 설명한다. 힌트와 AI 요청 예시는 필요한 경우에만 사용하며 모두 순서대로 소진하지 않는다. 이미 확인한 결론을 모르는 척하거나 질문 횟수를 채우지 않는다. 오류 분기는 실제 오류가 있을 때만 사용한다.

시작. 자료를 받아 작업 폴더 열기

[화면 03-START] ZIP 다운로드·압축 해제·VS Code에서 시작코드 폴더 열기·파일 탐색.
[조작] 배포된 01일차_실습3_기록추적.zip을 내려받아 작업 위치에 압축 해제한다. VS Code의 파일 → 폴더 열기로 01일차_실습3_기록추적/시작코드를 연다. ZIP 안에서 편집하지 않는다. 배포 출처를 확인하고 작업 공간 신뢰 여부를 판단한다. README.md와 recorded/case_a.json부터 연다. 함수는 규칙 탐색 단계에서, 테스트 기대값은 자기 기준을 정리한 뒤 확인한다.

기록 추적 실습 자료를 내려받겠습니다. 파일 이름이 세 번째 실습인지 확인하고 작업할 위치에 압축을 풀겠습니다. 이제 VS Code에서 파일, 폴더 열기를 선택합니다. 압축을 푼 폴더 안에 시작코드가 있습니다. 이 폴더를 열겠습니다.

탐색기에 자료와 파이썬 파일이 함께 보입니다. 먼저 설명 파일을 읽겠습니다. 오늘 만들 부분은 기록 사이의 참조를 비교하는 함수 하나입니다. 파일을 읽고 결과를 저장하는 부분과 테스트는 이미 제공되어 있습니다. 목표와 입력, 고칠 범위, 확인 방법이 정해져 있으므로 비교 규칙을 이해하고 함수 하나로 옮기는 데 집중하겠습니다.

저장 기록은 recorded 폴더에 있고, 제가 고칠 것은 trace_records 파일 안의 검사 함수입니다. 결과는 result 폴더에 저장됩니다. 별도로 들어 있는 코드 폴더는 비교용 완성본이므로 지금은 열지 않겠습니다.

검사 함수에는 아직 구현하지 않았다는 오류가 들어 있습니다. 이 상태로 실행하면 그 자리에서 멈추도록 만들어 둔 겁니다. 실제 설비 장애를 넣어 둔 것이 아닙니다. 먼저 무엇을 비교할지 정한 다음 이 빈 부분을 채우겠습니다.

먼저: 경보에서 작업지시까지 전체 과정 읽기

[화면 03-00] 동일 사건 기록 중 확보된 구간, 식별자·시각, 미확인 구간.
[조작] 전체 과정의 기록 종류를 짚되 제공 발췌가 모든 단계를 담고 있다고 전제하지 않는다. 원문 없는 구간을 다른 사건으로 채우지 않는다. 승인 전 명령 없음은 해당 구간의 기록 범위가 충분할 때만 판정한다.

참조를 비교하기 전에 전체 흐름에서 각 기록이 무엇을 말하는지 보겠습니다. 경보는 이상 조건이 성립했다는 기록이고, 추천은 어떤 조치를 제안했는지 보여 줍니다. 담당자의 승인, 설비에 보낸 명령, 처리 응답은 다시 구분해야 합니다.

팬 속도가 그대로였으면 승인 전 명령이 없었다고 말해도 될까요? 그것만으로는 부족합니다. 명령 기록과 그 기록이 포괄하는 시각을 확인해야 합니다. 응답을 받은 것, 유온이 완화된 것, 핀 세척이 끝난 것도 각각 다른 결과입니다.

제가 가진 자료에서 확인되는 구간을 먼저 적고, 없는 구간은 미확인으로 남기겠습니다. 오늘 만드는 도구는 이 흐름 전체의 성공을 판정하지 않습니다. 마지막 작업지시에서 판단을 거슬러 찾을 때, 직접

참조가 맞는지를 비교합니다.

1. 입력을 읽고 첫 질문 보내기

[화면 03-01] 원본 발췌의 직접 참조·도착 식별자, Copilot Chat에 선택한 입력과 첫 요청.
[조작] recorded/case_a.json에서 work_order.decision_id, decision.id, source_paths, provenance, context를 구분한다. VS Code에서 Copilot Chat을 열고 입력 JSON을 컨텍스트로 선택·첨부한다. 화면에 표시된 첨부 파일명을 확인한다. 해당 기능이 없으면 필요한 발췌를 붙여 넣고 원본 경로를 적는다. 개인 정보·비밀값을 포함하지 않는다.

이 자료는 이천이십육 년 시월 십사 일의 실제 실행에서 필요한 항목을 발췌한 것입니다. 교재의 예시 사건과는 별도 실행이므로 온도나 식별자를 서로 옮겨 채우지 않겠습니다.

파일을 위에서부터 읽어 보겠습니다. 원본을 설명하는 부분이 있고, 작업지시와 판단에 해당하는 묶음도 있습니다. 이름에 아이디가 붙은 항목이 여럿 보이는데, 이것들을 아무렇게나 이어도 되는 걸까요? 우선 각 항목이 자기 기록의 이름인지, 다른 기록을 가리키는 값인지 나누어 메모하겠습니다.

같은 설비라는 이유만으로 묶는 방법도 떠오릅니다. 그런데 같은 설비에 다음 날 다른 사건이 생기면 구분할 수 없겠네요. 제가 필요한 것은 비슷한 기록을 모으는 방법이 아니라, 이 작업지시가 어느 판단에서 나왔는지 찾는 방법입니다. 여기까지 파악한 내용을 질문에 넣겠습니다.

Copilot 채팅을 열고 지금 읽은 입력 파일을 붙이겠습니다. 제가 보고 있는 파일이 자동으로 모두 전달됐다고 생각하지 않고, 첨부된 파일 이름까지 확인합니다. 아직 비교할 항목을 확정하지 않았으므로 코드부터 요청하지 않겠습니다.

[입력 · 비낭독] 첫 요청: 첨부한 자료에서 작업지시가 어느 판단에서 나왔는지 찾고 싶습니다. 식별자가 여러 개라 각각 자기 기록의 이름인지 다른 기록의 참조인지 아직 구별 중입니다. 코드는 작성하지 말고, 제가 자료에서 먼저 찾아볼 항목을 질문 하나로 좁혀 주세요. 처음부터 비교할 필드와 정답을 모두 알려 주지는 마세요.
[응답 분기 · 비낭독] 실제 응답의 질문을 읽는다. 비교 대상 질문이면 아래 자기 답변을 사용한다. 다른 질문이면 입력에서 확인한 사실로 해당 질문에 답한다. 질문 없이 구현을 내놓으면 "코드는 아직 적용하지 않고 비교 대상부터 설명하겠습니다."라고 말하고 요청 범위를 다시 좁힌다. 질문·답변을 미리 받은 것처럼 꾸미지 않는다.

질문을 받은 부분으로 돌아가 보겠습니다. 작업지시에는 자기 식별자 외에 판단을 가리키는 항목이 있습니다. 그 값을 판단 묶음의 식별자와 나란히 놓겠습니다. 두 값이 같은지, 원본 위치도 맞는지 보겠습니다. 제가 찾은 비교 대상은 이 두 항목입니다.

제가 이렇게 찾은 이유를 채팅에 적겠습니다. 작업지시에 적힌 판단 참조가 도착한 판단의 식별자와 맞아야 같은 연결이라고 볼 수 있습니다. 단순히 같은 화면이나 같은 묶음에 있다는 이유만으로 연결하지 않겠습니다.

[입력 · 비낭독] 실제 질문에 대한 강사 답변과 위 판단을 채팅에 입력한다. source_paths의 원본 위치도 확인해 연결표에 기록한다.

2. 검사 함수 하나 구현하고 변경 검토하기

[화면 03-02] 시작 함수, 두 번째 요청, 실제 제안의 변경 비교, 저장.
[조작] trace_records.py를 컨텍스트에 추가한다. 자기 기준을 정리한 뒤 test_trace_records.py를 열어 검증 범위와 대조한다. 빈 check_link와 이를 호출하는 inspect_records, 출력에서 사용하는 반환 항목을 확인한다. 수정 대상은 시작코드의 함수 한 개다.

이제 함수를 열겠습니다. 비교 결과와 이유를 돌려주도록 자리는 마련되어 있지만, 어떤 조건으로 나눌지는 비어 있습니다. 가장 먼저 떠오르는 방법은 두 값이 같으면 확인, 다르면 실패로 나누는 겁니다. 그런데 자료가 부족할 때도 실패라고 해도 될까요? 기록이 잘못 연결된 것과 비교할 근거가 없는 것은

다른 상황 같습니다.

여기서 두 번째 입력을 잠깐 열겠습니다. 교육용으로 일부를 가린 자료입니다. 원본과 달라진 항목을 찾아 두 파일 옆에 적겠습니다. 없는 값을 다른 곳에서 가져와 채우지 않고, 지금 자료로 무엇까지 말할 수 있는지 정해야겠습니다.

[조작 · 비낭독] 두 JSON의 구조를 비교해 누락 항목을 직접 찾는다. 발견 전에는 해당 필드에 바로 커서를 놓아 답부터 보여 주지 않는다. 구조 비교가 막히면 "두 파일의 같은 위치를 나란히 보면 무엇이 달라졌나요?"라고 묻는다. 정답 필드에 커서를 놓거나 판정 규칙을 힌트로 읽지 않는다.
[입력 · 비낭독] 기준 탐색 요청: 두 값의 같음과 다름으로만 판정하려 했는데, 일부 항목이 없는 입력에서는 결론을 내릴 근거가 부족해 보입니다. 지금 제 기준이 놓치는 경우를 하나씩 질문해 주세요. 먼저 제가 경우를 나누어 설명한 뒤에 구현하겠습니다. 입력 자료는 바꾸지 마세요.
[응답 분기 · 비낭독] 실제 질문에 맞춰 강사가 입력의 값과 존재 여부를 확인한다. 답을 먼저 받은 경우에도 제안된 경우를 원본·가림 자료와 대조한 뒤 자기 말로 다시 정리한다. 실제 질문 없이 문답을 주고받은 것처럼 이어 읽지 않는다.

이제 세 경우를 나누겠습니다. 비교할 값이 없으면 미확인, 둘 다 있는데 다르면 실패, 둘 다 있고 같으면 확인으로 두겠습니다. 빈 문자열 두 개는 같다는 계산 결과가 나와도 연결을 입증하지 못합니다. 그래서 일치 여부보다 값이 있는지를 먼저 보겠습니다. 이 기준은 제가 자료를 읽고 정한 판단이고, 코드는 그 판단을 옮기는 역할입니다.

함수 아래를 보면 검사 결과를 모으고 화면에 출력하는 부분이 있습니다. 이 부분이 사용할 항목도 반환해야 합니다. 판정만 돌려주면 나머지 부분이 읽을 정보가 사라집니다. 호출 방식과 반환 형식을 유지하면서 함수 안만 구현해 달라고 요청하겠습니다.

[입력 · 비낭독] 구현 요청: 제 규칙은 참조 값 또는 도착 식별자가 없거나 빈 문자열이면 미확인, 둘 다 있고 다르면 실패, 둘 다 있고 같으면 확인입니다. 첨부한 시작본 trace_records.py의 check_link 함수만 구현해 주세요. 함수 인자와 나머지 입출력·테스트는 유지하세요. link, state, reference, identity, reason을 반환해야 합니다. recorded 파일과 context는 수정하거나 누락 참조를 추정하는 데 사용하지 마세요. 적용 전 변경 내용과 조건 순서를 설명해 주세요.
[응답 분기 · 비낭독] 실제 제안에서 값 추출·누락 검사·불일치 검사·반환 항목을 차례로 읽는다. 변경 비교에서 함수 밖 수정, 입력 변조, context 추정이 있으면 수락하지 않는다. 텍스트 제안이면 함수 부분만 원본과 나란히 비교한다. 필요한 지적만 후속 요청하고 다시 검토한다.

변경된 줄을 보겠습니다. 먼저 두 값을 어디에서 가져오는지, 그다음 누락을 먼저 확인하는지 보겠습니다. 마지막으로 판정과 이유, 비교한 두 값을 함께 돌려주는지 확인합니다. 입력 자료를 바꾸거나 다른 함수까지 고친 부분은 이번 요청 범위에 들어가지 않습니다. 테스트도 제 코드가 맞는지 확인하는 기준이므로 결과에 맞춰 고치지 않겠습니다.

[범위 이탈 분기 · 비낭독] 전체 구현을 바꾸거나 입력·테스트를 수정한 제안이면 적용을 멈추고 "요청보다 수정 범위가 넓어졌습니다. 입출력과 검사 기준은 그대로 두고, 빈 함수 하나로 범위를 돌리겠습니다."라고 말한다. 채팅에 입력·테스트·다른 함수 변경은 제외하고 check_link만 다시 제안해 주세요.를 보내고 변경 비교를 다시 확인한다.

[부분 수정 분기 · 비낭독] 실제 오류가 있으면 해당 줄과 이유를 말한 뒤 이 조건에서 빈 문자열 두 개가 확인으로 통과합니다. 값의 존재를 먼저 확인하도록 check_link 안의 해당 조건만 고쳐 주세요.처럼 실제 문제에 맞는 요청을 한다. 오류가 없으면 "정한 조건과 맞으므로 이 함수 변경을 적용하겠습니다."라고 말한다. 일부러 잘못된 응답을 연출하지 않는다. 검토한 변경만 적용하고 파일을 저장한다.

이 검사 코드는 로컬 파일만 읽으며 서버나 모델을 호출하지 않습니다. 지금 코드를 검토하는 Copilot 대화와 검사 프로그램의 실행은 별개입니다.

코드를 저장하겠습니다. 제안이 그럴듯한 것과 실제 입력에서 기대한 결과를 내는 것은 다릅니다. 이제 터미널에서 실행해 확인하겠습니다.

3. 첫 입력 실행하고 결과 해석하기

[화면 03-03] 시작코드에서 구현한 함수의 실행 출력과 새 결과 JSON.
[조작] VS Code 터미널을 연다. 현재 위치가 시작코드이고 trace_records.py, recorded가 그 아래 있는지 확인한다. python trace_records.py를 실행한다. result/trace_result.json을 연다. 폴더가 다르면 파일 탐색기 경로와 대조해 시작코드로 이동한다. 기존 완성본 result와 혼동하지 않는다.

터미널을 열고 현재 위치부터 확인하겠습니다. 지금 수정한 파일과 입력 폴더가 있는 시작코드 위치에서 실행해야 합니다. 첫 명령은 기본 입력을 읽어 결과를 저장합니다.

실행 전에 예상부터 적겠습니다. 원본 발췌에서는 작업지시의 판단 참조가 판단 식별자와 같습니다. 첫 연결은 확인이어야 합니다. 나머지 두 연결은 필요한 직접 참조나 독립된 사건 원문이 부족하므로 미확인으로 예상합니다.

[실행 결과 분기 · 비낭독] 실제 출력을 읽는다. 예상대로 확인 1개·미확인 2개이면 아래 해석을 이어 간다. NotImplementedError면 저장한 파일과 실행 경로, 빈 함수가 남았는지 확인한다. 파일 없음이면 작업 폴더와 입력 경로를 확인한다. 판정이 다르면 해당 연결의 입력값과 함수 조건을 대조하고, 실제 출력·예상·관련 함수만 채팅에 붙여 한 조건의 원인을 질문한다. 수정 후 같은 명령으로 다시 확인한다. 오류가 난 상태에서 아래 정상 결과 발화를 읽지 않는다.

첫 연결은 확인이고, 나머지 두 연결은 미확인입니다. 결과 파일의 두 값을 입력과 다시 대조하겠습니다. 전체가 미확인인 것은 프로그램이 멈췄다는 뜻이 아닙니다. 현재 자료만으로 모든 연결을 입증하지 못했다는 판정입니다.

context에 사건과 경보의 식별자가 함께 있더라도 없는 직접 참조를 대신 만들 수는 없습니다. 다음에 필요한 것은 판단이 사건을 가리키는 항목, 그리고 사건이 경보를 가리키는 항목을 확인할 수 있는 원문입니다. 제공 자료에서 못 찾은 것과 실제 시스템에 연결이 없다는 결론은 구분하겠습니다.

4. 두 번째 입력과 경계 조건으로 응용하기

[화면 03-04] 교육용 가림 입력의 예상, 두 번째 결과, 네 테스트, 필요한 부분 수정.
[조작] recorded/case_a_redacted.json을 열어 첫 입력과 비교한다. 이어 터미널에서 python trace_records.py recorded/case_a_redacted.json --output result/redacted_result.json, python -m unittest -v를 순서대로 실행한다. 두 번째 입력을 첫 결과 경로에 덮어쓰지 않는다.

두 번째 입력에서는 작업지시의 판단 참조 하나를 가렸습니다. 교육용으로 만든 파생 자료이지 실제 시스템에서 참조가 누락된 사례는 아닙니다. 이번에는 첫 연결도 미확인으로 바뀌어 세 연결 모두 미확인이어야 합니다. 예상이 정해졌으니 다른 파일 이름으로 결과를 저장하겠습니다.

[결과 분기 · 비낭독] 실제 가림 결과를 읽고 원본 결과와 비교한다. 첫 연결이 확인이라면 누락을 우회하거나 context에서 채우는 코드가 있는지 점검한다. 실제 차이를 첨부해 원본은 첫 연결 확인, 가림 입력은 미확인을 예상했습니다. 실제 차이를 보고 제 조건 중 어느 부분을 확인할지 질문해 주세요. 입력 자료는 고치지 마세요.라고 요청하고 필요한 함수 부분만 수정·저장·재실행한다. 예상과 맞으면 그대로 비교를 정리한다.

다음은 네 가지 입력을 자동으로 확인하는 테스트입니다. 원본에서 한 연결만 입증되는지, 다른 판단 식별자는 거부하는지, 참조가 없을 때 주변 정보로 추정하지 않는지, 빈 문자열 두 개를 연결로 인정하지 않는지 확인합니다.

다른 식별자를 넣은 경우에는 연결 판정이 실패여야 합니다. 그 실패를 예상대로 찾아내면 테스트는 통과합니다. 연결이 실패한 것과 테스트 자체가 실패한 것은 서로 다릅니다.

[테스트 분기 · 비낭독] 네 테스트의 실제 통과·실패를 읽는다. 실패가 있으면 그 테스트 하나의 기대값·실제값과 함수만 첨부하고 원인 설명을 요청한다. 강사가 설명을 검토한 뒤 필요한 조건만 수정·저장하고 두 실행 명령과 테스트를 다시 확인한다. 모두 통과하면 "정해 둔 네 입력에서 기대한 판정이 나왔습니다."라고 말한다. 통과 개수·응답 내용은 실제 결

과를 따른다.

이 검사는 정해 둔 입력에서 코드가 어떤 판정을 내리는지 확인합니다. 테스트가 통과해도 자료에 없는 사건 연결까지 입증된 것은 아닙니다. 코드의 동작과 사건의 근거를 별도로 적겠습니다.

5. 판단의 한계까지 정리하기

[화면 03-05] 연결표, 직접 구현한 코드, 두 결과, 테스트 관측, 추가로 필요한 원문.
[조작] 실습 폴더의 연결표.md를 열어 비교 필드·값·판정·원본 위치를 정리한다. 구현한 시작코드와 그 result를 연결한다. 실제 관측한 결과만 적고 미해결 항목을 남긴다.

마지막으로 연결표를 정리하겠습니다. 비교한 필드와 값, 판정, 원본 위치를 함께 남깁니다. 두 입력에서 어떤 참조가 달라졌고 결과가 어떻게 바뀌었는지도 적겠습니다.

제공된 작업지시는 배정됨 상태이고 완료 시각은 없습니다. 이 도구가 확인하는 것은 참조의 연결입니다. 명령 응답이나 유온 완화, 실제 정비 완료를 대신 판정하지 않습니다.

근거가 부족한 곳을 찾아 미확인이라고 설명한 것도 이번 검토의 결과입니다. 전체 추적을 완료했다고 바꾸기보다 어디까지 확인했고 어떤 원문이 더 필요한지 남기겠습니다. 이제 그 판단에 쓰이는 지식과 원인별 조치의 관계를 살펴보겠습니다.

[연결 · 비낭독] 이론 46장에서 재개한다. 화면 03-START·03-00~05는 후속 삽입한다. 문서 완성과 새 구현의 실행 검증·전체 사건 입증을 구분한다.

실전반 1회차 — 관계그림 실습 대본

[운영 · 비낭독] 강사가 혼자 자료를 읽고 조작·판단·요청·검토를 시범 보이는 발화다. 학생 응답이나 조작을 기다리지 않는다. 이번 문서 제작에서는 실습·화면 조작·캡처를 하지 않는다. 화면은 가이드의 동일 ID 자리에 나중에 넣는다. 아래 조작 안내는 수행 완료 기록이 아니다. 운영 시간은 미확정이다. 이전 고정 견적은 철회했다. 현재 도구의 저장·재열기 동작은 후속 화면을 넣을 때 확인한다. 그림은 새 고장 실행이나 스키마 검증의 완료본이 아니다.

[진행 원칙 · 비낭독] 조작하면서 해당 판단을 설명한다. 힌트와 AI 요청 예시는 필요한 경우에만 사용하며 모두 순서대로 소진하지 않는다. 이미 확인한 결론을 모르는 척하거나 질문 횟수를 채우지 않는다. 오류 분기는 실제 오류가 있을 때만 사용한다.

시작. 자료를 받아 작업 폴더 열기

[화면 04-START] 실습 자료 다운로드 → 압축 해제 → VS Code 폴더 열기 → 탐색기. 화면은 후속 삽입 자리다. 다운로드 링크는 배포한 01일차_실습4_관계그림.zip을 연결한다. 실제 실습·캡처는 이번 문서 제작에서 수행하지 않는다.
[조작] 이전 실습 파일을 닫는다. 수업 자료의 해당 ZIP을 내려받고 파일 탐색기에서 압축을 푼다. VS Code를 열어 파일 → 폴더 열기로 01일차_실습4_관계그림을 선택한다. 출처를 확인한 자료만 작업 공간 신뢰 여부를 판단한다. 파일 하나만 열거나 ZIP 내부에서 편집하지 않는다.

이번 실습 ZIP을 내려받아 새 작업 폴더에 압축을 풀겠습니다. VS Code의 파일, 폴더 열기에서 그 폴더를 선택합니다. 탐색기에서 실습 이름을 확인하고 관계_검토표.md를 열겠습니다. 이번에 작성할 항목과 결과를 저장할 위치부터 보겠습니다.

[조작] 읽기 전용 입력과 수정할 파일, 결과가 저장될 위치를 차례로 짚는다. 전체 경로에 개인 사용자 이름이 보이면 화면에서 제외한다.

검토표에는 대상과 관계를 적을 자리가 있습니다. 어떤 종류의 대상을 구분할지는 이 틀을 따라가고, 관계의 이름과 방향은 제가 교재 근거를 읽고 정하겠습니다. 그림 전체의 정답을 먼저 받아 옮기지는 않겠습니다. 필요한 부분을 읽고 실제 작업 도구로 이동하겠습니다. 작업을 마친 뒤에는 이 폴더로 돌아와 근거와 결과를 정리하겠습니다.

1. 대상과 근거 분리

[화면 04-01] 대상 목록과 매뉴얼 근거 대목.
[조작] 교재 64~71장과 HM-8.1·8.4·8.5·8.7을 준비한다. 설비·구성 요소·센서·증상·고장 유형·원인·증거·조치를 나누고 관측값과 판별 기준을 구분한다.

이번에는 앞에서 본 설비 이상을 관계 그림으로 정리하겠습니다. 매뉴얼을 열고 대상 이름과 상태를 나타내는 표현부터 찾아 적겠습니다. 처음부터 화살표를 그리지 않고, 무엇을 서로 구분해야 할지 보겠습니다.

유온 상승과 핀 오염이 보입니다. 둘 다 이상을 설명하는 말인데 같은 종류로 놓아도 될까요? 원문에서 어떤 현상이 나타났다고 하는지, 무엇을 원인으로 확인한다고 하는지 다시 읽겠습니다. 유온 상승은 관측한 증상이고 핀 오염은 그 원인 후보로 나누겠습니다. 이렇게 나누어야 무엇을 보았고 무엇을 판단했는지 구별할 수 있습니다.

[조작 · 비낭독] 배포한 참고_HM-8_매뉴얼.md의 HM-8.4를 읽고 직접 대상 목록을 적는다. 완성 대상 목록을 먼저 붙여 넣지 않는다. 분류가 모호한 항목 하나를 고른 뒤 필요한 문장만 채팅에 첨부한다.
[입력 · 비낭독] 분류 탐색 요청: 이 문장에 나온 말들을 대상과 상태, 원인으로 나누어 보려 합니다. 제가 임시로 나눈 목록을 붙였습니다. 한 항목을 왜 그렇게 분류했는지 먼저 물어봐 주세요. 답한 뒤 원문에서 다시 읽을 부분을 알려 주세요. 완성 관계도나 전체 정답 목록은 만들지 마세요.

고장 유형은 어떤 기능에 문제가 생겼는지를 나타냅니다. 여기서는 쿨러 냉각 성능 상실입니다. 같은 기능을 잃더라도 핀이 막힌 경우와 팬이 제대로 돌지 않는 경우는 원인이 다를 수 있습니다. 이 차이가 그림에 드러나게 만들겠습니다.

문서에서 가져온 판별 기준과 실제 사건에서 관측한 숫자도 따로 적겠습니다. 기준은 무엇과 비교할지 정하고, 관측값은 그 사건이 조건에 해당하는지 판단할 때 씁니다. 숫자를 적는 것만으로 원인이 확정되는 것은 아닙니다.

2. 기본 관계 그리기

[화면 04-02] 이름과 방향이 보이는 핀 오염 기본 관계도.
[조작] arrows.app에서 노드를 만들고 실제 대상 이름을 붙인다. 교재의 관계 이름과 방향에 맞춰 연결한다. 줄긋기나 이름 편집의 구체 조작은 후속 촬영 시 실제 화면으로 확인한다.

브라우저에서 arrows.app을 열고 새 그림을 시작하겠습니다. 우선 노드 하나에 설비 이름을 적고, 다른 노드에는 쿨러를 적겠습니다. 두 대상의 관계를 문장으로 읽은 뒤 선을 연결하겠습니다. 선을 먼저 긋고 이름을 끼워 맞추지 않겠습니다.

[조작 · 비낭독] 새 그림에서 노드 추가→이름 편집→두 노드 연결→관계 이름 편집을 차례로 보여 준다. 화면의 현재 기능에 맞춰 클릭한다. 관계를 선택해 출발·도착을 확인하고 잘못 연결했을 때만 방향을 고친다.

대상을 놓은 다음 관계에 이름과 방향을 붙이겠습니다. 설비와 구성 요소, 구성 요소와 센서부터 연결하겠습니다. 쿨러는 설비의 구성 요소이고, 유온 센서는 쿨러의 상태를 계측하는 데 쓰입니다.

진단 쪽에는 앞에서 배운 관계를 적용하겠습니다. 원인을 찾는 순서와 화살표의 뜻을 구분했죠. 원문에서 두 대상이 연결된 문장을 찾아 읽고, 그 뜻에 맞춰 이름과 방향을 정하겠습니다.

[조작 · 비낭독] 교재와 매뉴얼의 근거 문장을 찾아 직접 관계를 그린다. 뜻이나 대상 종류가 불분명할 때만 해당 문장과 자신의 해석으로 질문한다. 이미 설명할 수 있는 방향을 모르는 것처럼 연출하지 않는다.

원문에서 확인한 뜻은 핀 오염이 냉각 성능 상실을 일으킨다는 것입니다. 그래서 원인을 출발점으로 놓겠습니다. 유온 상승은 그 고장을 가리키는 증상으로 연결하겠습니다. 출발 대상을 주어로 읽으면 관계의 뜻을 확인할 수 있습니다. 같은 선이라도 이름이나 방향을 바꾸면 문장이 달라집니다.

화살표가 자연스럽게 보인다는 것만으로 맞다고 판단하지 않겠습니다. 관계 이름의 뜻, 양 끝 대상의 종류, 문서의 근거를 함께 확인해야 합니다. 고장 유형에서 원인 쪽으로 거슬러 찾아보는 일과 저장된 화살표를 뒤집는 일도 다릅니다. 찾아가는 방향을 바꿔도 사실의 뜻은 유지해야 합니다.

3. 핀 오염 증거 연결

[화면 04-03] 핀 오염의 두 증거 조건, 완화·근본 조치와 출처.
[조작] CE 최근 30초 평균 < 70 %, VS1 최근 2분 평균 < 0.9 mm/s를 HM-8.4와 대조해 연결한다. 팬 최대 운전과 핀 세척의 문서 근거를 구별한다.

핀 오염을 판단하려면 두 조건을 함께 봅니다. 냉각 효율은 최근 삼십 초 평균이 칠십 퍼센트 미만인지, 팬 진동은 최근 이 분 평균이 초당 영 점 구 밀리미터 미만인지 대조합니다. 화면에서 각 값의 구간과 단위를 함께 확인하겠습니다.

경보가 났을 때 한 번 측정한 값과 진단에 쓰는 구간 평균은 서로 바꾸어 쓰지 않습니다. 평균이라고만 적어도 부족해요. 어느 값을 어느 구간에서 읽었는지가 남아야 같은 기준으로 검토할 수 있습니다.

조치도 나누어 그리겠습니다. 팬 최대 운전은 우선 유온을 낮추는 완화 조치이고, 핀 세척은 오염을 없애는 근본 조치입니다. 팬을 돌린 뒤 온도가 내려갔다는 관측으로 핀 세척까지 끝났다고 말할 수는 없겠죠. 조치의 관계와 근거 절을 각각 붙이겠습니다.

4. 팬 구동 불량 가지 추가

[화면 04-04] 공통 고장 유형과 두 원인의 증거·조치·출처.
[조작] 기존 그림을 유지하며 팬 구동 불량 가지를 추가한다. VS1 최근 2분 평균 ≥ 0.9 mm/s는 HM-8.4, 팬 점검은 HM-8.1에서 확인한다. 순간값 문장과 진단용 구간 평균 기준을 섞지 않는다.

공통 고장 유형을 유지하면서 팬 구동 불량이라는 다른 원인 가지를 더하겠습니다. 이번에는 팬 진동의 최근 이 분 평균이 같은 기준 이상인지 봅니다. 팬 쪽 문제를 가리키는 조건이면 문서의 팬 점검으로 이어집니다.

같은 냉각 성능 상실에 연결됐다고 핀 세척과 팬 점검을 서로 바꾸어도 되는 것은 아닙니다. 조치가 어느 원인에 맞는지를 문서에서 확인해야 합니다. 그림에서 팬 구동 불량을 따라갔는데 바로 핀 세척으로 이어진다면 그 연결을 다시 살펴보겠습니다.

이 가지는 매뉴얼에 있는 다른 원인 경로를 표현한 것입니다. 앞 사건에서 진동이 낮았다는 관측을 팬 구동 불량의 발생 증거로 사용할 수는 없습니다. 두 원인을 비교하는 그림을 만든 것과 실제 사건에서 두 고장이 모두 확인됐다는 말은 다릅니다.

5. 방향 검토와 저장

[화면 04-05] 편집 파일·이미지와 재열기 결과.
[조작] 관계 한 개의 출발·도착·이름·출처를 대조한다. 편집 파일과 이미지로 저장한 뒤 편집 파일을 다시 열어 두 원인과 출처가 남았는지 확인한다. 확인 전에는 결과란을 채우지 않는다.

두 원인의 가지를 나란히 보겠습니다. 같은 고장 유형에 연결되지만 증거 조건과 조치가 다릅니다. 두 번째 가지를 만들면서 첫 번째의 조건이나 조치를 그대로 가져온 곳이 없는지 원문과 비교하겠습니다. 차이를 설명할 수 있으면 검토표에 근거를 남기겠습니다.

[조작 · 비낭독] 두 가지의 증거 항목·구간·비교 방향·조치·출처를 대조한다. 관계 방향은 작성할 때 확인한 기준을 유지하는지 검사한다. 차이를 설명할 수 없는 항목이 있을 때만 실제 의문과 원문을 AI에 전달한다. §2에서 해결한 같은 관계를 다시 질문하지 않는다. 오류가 없다면 검토 근거를 남기고 저장한다.

고칠 곳이 있다면 그 관계만 수정하고 다시 문장으로 읽겠습니다. 검토를 마쳤으면 저장하겠습니다.

완성 그림은 이미지와 편집 파일로 함께 남기겠습니다. 이미지는 결과를 보여 주고, 편집 파일은 다음 회차에 관계를 고치는 데 씁니다. 다시 열었을 때 두 원인과 각각의 증거·조치·출처가 남아 있는지 확인하겠습니다.

이 그림은 다음 회차에 지식의 구조를 구체화하는 입력으로 쓰겠습니다. 이제 교재로 돌아가, 오늘 살펴본 경보와 판단, 승인과 실행 기록을 구별하며 복습하겠습니다.

[연결 · 비낭독] 같은 이론③ 72장에서 재개한다. 화면과 재열기 관측은 후속 삽입하며, 그림 작성만으로 진단 실행·스키마 검증을 완료했다고 표시하지 않는다.